Tableau — One-Pager

> *The Bloomberg Terminal for the 600 restaurants where access is the product.*

The user

Priya Shah, 31. Business development at a venture firm. Upper East Side. She books six special-occasion dinners a year — anniversaries, client wins, friend birthdays for groups of six. She has Resy, OpenTable, Tock, and Yelp Reservations on her home screen. Last month she spent **roughly four hours refreshing them** and still ate at her second-choice restaurant on her own birthday.

There are ~50,000 people like Priya in NYC alone: 28-to-45-year-old professionals making $200K+ who treat the hardest reservations as social signaling and would happily pay $19/month to outsource the refresh-and-pray ritual. Tableau is for them. Not for the long tail of casual diners — Resy already serves those well.

The problem

Priya's behavior today:

1. Sets a phone alarm for 9:00 AM exactly 30 days before the date she wants. Refreshes the Carbone calendar at 10:00 AM ET. Loses to bots and scalpers within 8 seconds.
2. Joins the Resy notify list, gets one email a week saying "a slot opened" — but only after it's already gone.
3. Buys reservations on Appointment Trader for $50–$500 each. Appointment Trader sold $80M+ of reservations last year, almost all to people like Priya. **The willingness-to-pay is proven.**

What she actually needs: - Continuous monitoring she doesn't have to think about. - A *judgment layer* — most "open slots" don't fit (wrong time, wrong size, wrong vibe). She wants the ones that fit her. - One tap to confirm.

The product

A multi-agent system on LangGraph. Six nodes across two graphs:

1. **Supervisor** (chat) — parses "watch Don Angie next Friday for 2" into a structured `Watch` with `auto_book=True`.
2. **Scout** (tick) — every 2 minutes, polls availability and hash-diffs against the last snapshot. **95% of ticks short-circuit on the hash, never calling an LLM.** This is the lever that makes the unit economics work.
3. **Ranker** (tick) — when a slot opens, retrieves restaurant context (RAG over food-media editorial) plus user history and writes a 2-sentence "why this fits."
4. **Auto-Booker** (tick) — opens a real Chromium browser via Playwright, navigates the booking platform, clicks the time, fills the form, submits, captures the confirmation page. The agent really books.
5. **Notifier** (tick) — fires an in-app card + a real email via Resend with the confirmation code and a link to the confirmation page.
6. **Booker** (chat, fallback) — HITL gate for one-off bookings outside an active watch.

Live demo: https://concierge-web-eheg65qzwa-uc.a.run.app

Unit economics — one user-month

Assumes 4 active watches, 2-minute polling cadence, 30-day window.

ITEM	VOLUME	COST
Scout polling — 95% short-circuit on hash; 5% trigger Gemini Flash (300 in / 50 out)	~4,300 LLM calls	**$0.85**
Ranker (Gemini Flash, 2K in / 400 out) on 8 slot-opens	8	$0.013
Supervisor + chat (Gemini Flash, 5 turns, 3K in / 600 out)	5	$0.012
Browser-automated booking (Playwright, no LLM)	2 successful books	$0.00
Notifier (Gemini Flash, 1K in / 200 out)	8	$0.005
Embeddings (Vertex `text-embedding-005`, amortized)	one-time	$0.01
Cloud Run + Firestore (amortized over 100 users)	—	$0.40
Email (Resend, 3,000/mo free)	8	$0.00
Total COGS		**~$1.30**

LLM cost is **$0.88 / user-month** (~5% of revenue) because we run on Gemini 2.5 Flash via Vertex AI — billed against our GCP credits and an order of magnitude cheaper per token than the Anthropic equivalent. The hash-diff Scout still does the heavy lifting (polling is 99% of call volume, but only 5% of calls cost LLM tokens).

Pricing - **Concierge $19/mo** — 4 watches, in-app + email - **Concierge+ $49/mo** — 10 watches, priority queue, SMS

Gross margin at $19: **93%**. Breakeven CAC at 6-mo payback: ~$106. Aspirational LTV at 18-mo retention: ~$320.

Where it breaks (named honestly)

1. **Resy / OpenTable cease-and-desist.** Most likely outcome of any unauthorized scraping. **Mitigation** built into the architecture: every provider call sits behind a `ReservationProvider` interface; the demo runs on a `MockResyProvider` and the agent books against a Tableau-controlled sandbox (`TableTime`). The path to real bookings is partnership, not adversarial scraping. We have a B2B pivot pre-built (below).
2. **Polling cadence pressure.** If a competitor offers sub-30-second polling, costs scale 4× and margin compresses to ~70%. Still profitable, but the moat shifts from raw frequency to ranker quality.
3. **Cold-start.** No users → no behavioral signal → the Ranker reduces to defaults. We mitigate by launching with a partner concierge desk so the Ranker has multi-user behavioral data on Day 1.
4. **Vertex / Gemini price moves or quota cuts.** The LLM client wrapper (`backend/llm/client.py`) is provider-agnostic — `LLMResponse` is a unified shape and switching to Anthropic, OpenAI, or self-hosted only changes the implementation of `chat()`. A 3× Vertex hike cuts margin to ~80%; we'd absorb it.

Path forward

White-label API → luxury hotel concierge desks (Aman, 1 Hotels, Faena, Mark Hotel). Tested estimate: **$5,000/mo per property × 50 properties → $3M ARR with one BD hire.** This pivot solves both cold-start (hotels supply behavioral data) and ToS risk (hotels have direct restaurant relationships). The consumer subscription is the wedge; concierge B2B is the business.

Why these technical choices

CHOICE	HOW IT SERVES THE USER, PROBLEM, OR ECONOMICS
Multi-agent (LangGraph)	The user's request ("watch this") naturally decomposes into specialists: parse, poll, rank, book, notify. A single-prompt agent would re-do the parsing every tick — that's the cost trap multi-agent avoids.
Hash-diff Scout that mostly skips the LLM	The single biggest lever in our economics. Polling is the dominant call volume; making 95% of polls free changes the unit economics by an order of magnitude.
Gemini 2.5 Flash via Vertex AI	Billed against GCP credits — zero out-of-pocket while we iterate. ~10× cheaper per token than Anthropic Sonnet, and the unified `LLMResponse` abstraction lets us swap providers in one env var if pricing moves.
Firestore (incl. vector search) instead of Postgres + pgvector + Memorystore	One service, single-digit-cents scaling, no minimum spend. Replaces three GCP services that would cost ~$50/mo idle. Local-JSON fallback for dev so the agent stack boots without enabling APIs.
Cloud Run min-instances=0	Idle cost approaches zero. We hit 93% gross margin from user 1.
Auto-book at watch creation, not at slot open	Consent is given when the user creates the watch with explicit date/party/time. The agent then books any matching slot without re-prompting. UX win + legal posture (we book exactly what the user opted into, nothing else).
Browser automation against a Tableau-owned sandbox (`TableTime`)	The agent really completes the booking — opens Chromium, fills the form, captures the confirmation. We drive a site we own so no broken selectors and no ToS exposure. Same architecture a Resy partnership would use; we flip `USE_FAKE_RESY=false` and the agent code is unchanged.
`MockResyProvider` behind a `ReservationProvider` interface	Production runs on mock data. The day a partnership exists, we flip a flag. This is the architectural answer to the regulatory question.

What we're asking for

A path to launch with one luxury-hotel concierge partner. We have the agent stack, the pricing model, and a defensible architectural answer to the regulatory question. We need 90 days and a single hospitality-side relationship to validate B2B pricing before opening the consumer flywheel.

Built by Myriam Bengoechea Pardo (`mb5500`) and Blanca Valera Caballero (`bv2358`) for IEORE4576 — Agentic AI for Analytics, Columbia, Spring 2026. GitHub: https://github.com/myriaambp/reservation-concierge.